
TP — Environnements Virtuels Collaboratifs

Rapport écrit du Lab

GEORGI Thomas

thomas.georgi@imt-atlantique.net

KI Alidou

alidou.ki@imt-atlantique.net

Dépot Github : <https://github.com/thomasgrgi/TP-Multiplayer>

IMT Atlantique
Semestre printemps 2026

Contents

1 Introduction

2 Mise en place de la Navigation et de l'Architecture (Étapes 1 & 2)

2.1 Gestion de la Caméra et Autorité

3 Interaction et Conscience Partagée - Awareness (Étapes 3 & 4)

3.1 Synchronisation des états interactifs

3.2 Transfert de Propriété et RPC

4 Intégration de la Réalité Virtuelle (Étape 5)

4.1 Adaptation de la connexion

5 Améliorations, Difficultés Rencontrées et Solutions

5.1 Création d'une console de débogage in-game (Ajout)

5.2 Synchronisation de l'Avatar VR (Ajout)

5.3 Le bug du "Double Offset" (Erreur et Correction)

6 Conclusion

1 Introduction

Ce document présente le travail réalisé dans le cadre du TP portant sur la création d'un univers 3D partagé et immersif (Desktop et Réalité Virtuelle) sous Unity. L'objectif principal était de mettre en place une architecture réseau basée sur l'autorité distribuée (*Distributed Authority*) à l'aide de *Netcode for GameObjects*, puis d'intégrer des mécaniques d'interaction (XR Interaction Toolkit) synchronisées entre les clients. Enfin, le projet a été étendu pour accueillir des clients en Réalité Virtuelle (casque Meta Quest) interagissant dans le même environnement partagé qu'un utilisateur sur PC.

2 Mise en place de la Navigation et de l'Architecture (Étapes 1 & 2)

La première étape a consisté à configurer le projet multi-utilisateurs. Plutôt que d'utiliser un simple cube se déplaçant de manière basique, nous avons conçu un `NetworkedPlayer` évolutif.

2.1 Gestion de la Caméra et Autorité

Dans un environnement multijoueur, chaque client instancie un avatar. La difficulté réside dans le fait de ne pas s'approprier les caméras des autres joueurs. Nous avons modifié le `PlayerController` pour qu'il ne capture et n'active la caméra de la scène que s'il s'agit du joueur local (`IsOwner` et `IsLocalPlayer`). Cela garantit que les inputs de navigation (déplacement avant/arrière et rotation via le clavier) ne s'appliquent qu'à notre propre entité réseau.

3 Interaction et Conscience Partagée - Awareness (Étapes 3 & 4)

Afin de permettre au client PC d'interagir avec le monde, nous avons implémenté un curseur 3D contrôlable à la souris, évoluant en profondeur via la molette, actif lorsque la touche Alt est enfoncée.

3.1 Synchronisation des états interactifs

Lorsqu'un utilisateur manipule un objet partagé (ex: un `SharedCube`), il est crucial de fournir un retour visuel à la fois pour lui-même et pour les autres utilisateurs (*awareness*). Nous avons mis en place des changements de couleur : cyan lors du survol (*Hover*) et jaune lors de la saisie (*Select*).

3.2 Transfert de Propriété et RPC

Pour manipuler la physique d'un objet en autorité distribuée, le client doit en être le propriétaire. Nous avons implémenté un système où la méthode `LocalCatch` demande l'ownership au serveur via une méthode RPC (*Remote Procedure Call*). Une fois l'objet saisi, les changements visuels sont propagés sur le réseau à l'aide de l'attribut `[Rpc(SendTo.Everyone)]`. Cela permet d'appeler des méthodes comme `ShowCaughtRpc` sur toutes les instances de l'objet à travers le réseau, assurant ainsi une cohérence visuelle globale sans surcharger la bande passante avec des synchronisations de variables continues.

4 Intégration de la Réalité Virtuelle (Étape 5)

L'objectif final était d'introduire un client VR de manière asymétrique (Desktop vs VR). Nous avons créé une `VRScene` contenant un *XR Origin* transformé en prefab `VRPlayer` muni de composants `NetworkObject` et `NetworkTransform`.

4.1 Adaptation de la connexion

L'interface graphique classique (`OnGUI`) étant inadaptée à la VR, nous avons développé un script `VRConnectionManager`. La connexion au serveur ne se fait plus via un bouton 2D, mais en interagissant spatialement avec un objet 3D muni d'un composant *XR Simple Interactable*. Un simple déclenchement sur cet objet invoque la fonction `ConnectToSharedWorld()`. Pour permettre la communication, le client VR (casque en Wi-Fi) et le client PC (Hôte) ont été configurés pour utiliser la même adresse IP locale via le `UnityTransport`, remplaçant le `localhost` par défaut.

5 Améliorations, Difficultés Rencontrées et Solutions

Au-delà du sujet initial, nous avons pris l'initiative d'ajouter des fonctionnalités essentielles pour l'expérience utilisateur et le débogage, ce qui a soulevé certains défis techniques intéressants.

5.1 Création d'une console de débogage in-game (Ajout)

Ne disposant pas du Quest Link pour observer la console Unity en temps réel lors de l'exécution en VR, nous avons développé un `VRLogViewer`. Il s'agit d'un *Canvas* en espace monde (World Space) attaché au joueur, qui s'abonne à l'événement `Application.logMessageReceived`. Ce système intercepte tous les `Debug.Log` et les affiche sur un *TextMeshPro* dans le casque, facilitant grandement le débogage réseau.

5.2 Synchronisation de l'Avatar VR (Ajout)

Nous souhaitions que le joueur PC puisse voir les mouvements de tête et de mains du joueur VR en temps réel. Le `NetworkTransform` de base ne gérant que la racine du joueur, nous avons créé le script `VRAvatarSync`. Ce script utilise des `NetworkVariable` (`Vector3` et `Quaternion`) en permission d'écriture exclusive pour le propriétaire (`Owner`). Le joueur VR lit les capteurs locaux et écrit dans ces variables, tandis que le joueur PC lit ces variables pour animer des cubes visuels représentant le joueur VR.

5.3 Le bug du "Double Offset" (Erreur et Correction)

Problème rencontré : Bien que la synchronisation de l'avatar fonctionnait, le joueur PC voyait la tête et les mains du joueur VR flotter à plus de 3 mètres du sol, avec des comportements de rotation erratiques, alors que l'affichage était correct dans le casque.

Analyse : Ce problème découlait du comportement asymétrique du composant *XR Origin*. En VR, le `Camera Offset` est à 0 et la caméra monte physiquement à la taille du joueur (ex: 1m70). Sur PC (sans casque détecté), Unity relève artificiellement le `Camera Offset` à 1m60 pour simuler une hauteur humaine. Nos cubes visuels étant enfants de ce `Camera Offset`, le PC additionnait la hauteur simulée (1.6m) aux coordonnées réseau

regues du casque (1.7m), créant un double décalage vertical.

Solution apportée : Nous avons restructuré le prefab `VRPlayer` en plaçant les visuels (tête et mains) à la racine de l'objet, en dehors des sous-systèmes de caméra. Dans le code, nous avons remplacé la lecture des positions locales par des mathématiques relatives globales via `InverseTransformPoint` :

```
1 // Calcul des positions relatives à la racine du VRPlayer,  
2 // ignorant les offsets internes du système XR.  
3 headPos.Value = transform.InverseTransformPoint(vrHead.position);  
4 headRot.Value = Quaternion.Inverse(transform.rotation) * vrHead.rotation  
    ;
```

Cette approche garantit que les données transitant sur le réseau représentent la distance absolue entre le sol et les membres du joueur, offrant une représentation 1:1 parfaite sur le client PC.

6 Conclusion

Ce projet nous a permis d'appréhender concrètement les défis liés au développement multijoueur asymétrique. La gestion de l'autorité, la distinction entre la logique locale et réseau (via les RPCs), et la gestion des espaces de coordonnées divergents entre les interfaces Desktop et VR ont été des éléments centraux et formateurs de ce TP.